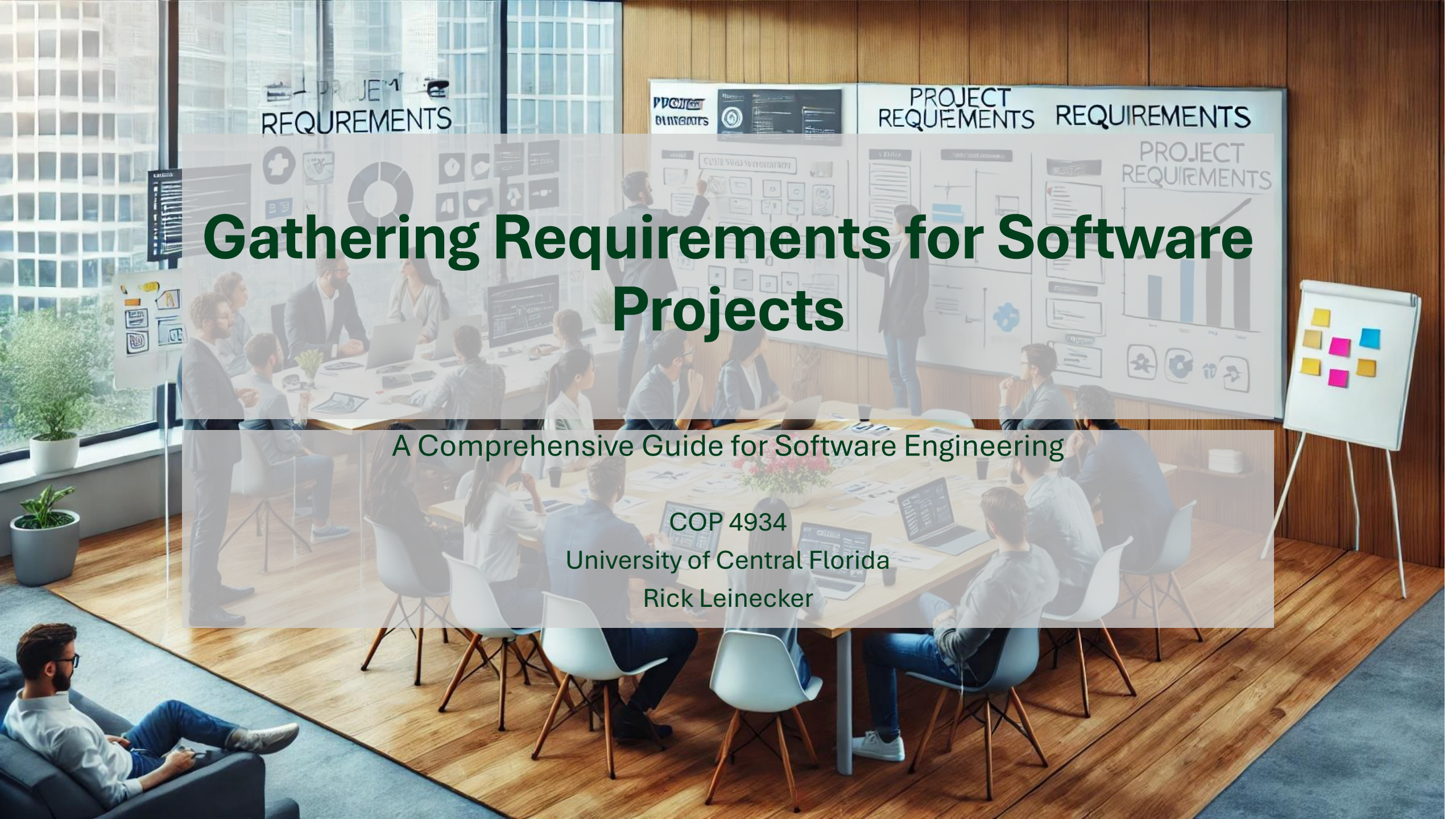


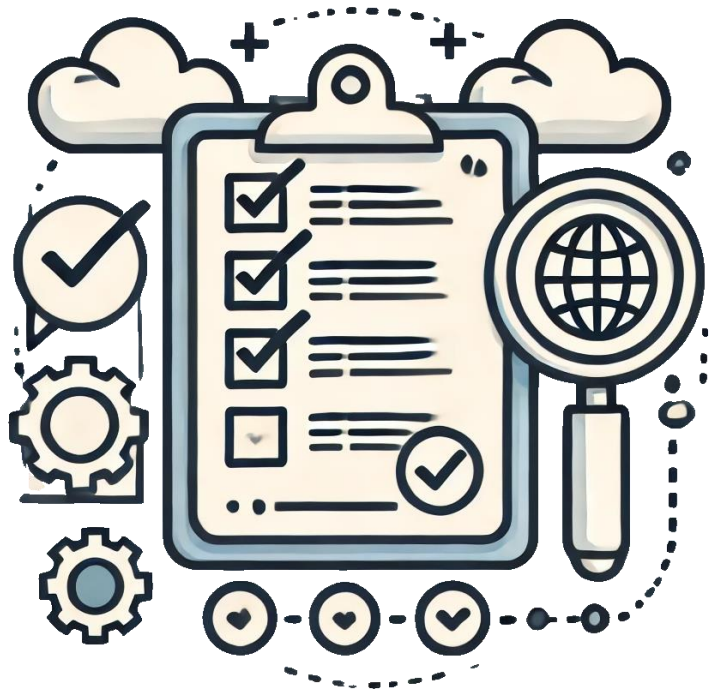
Gathering Requirements for Software Projects

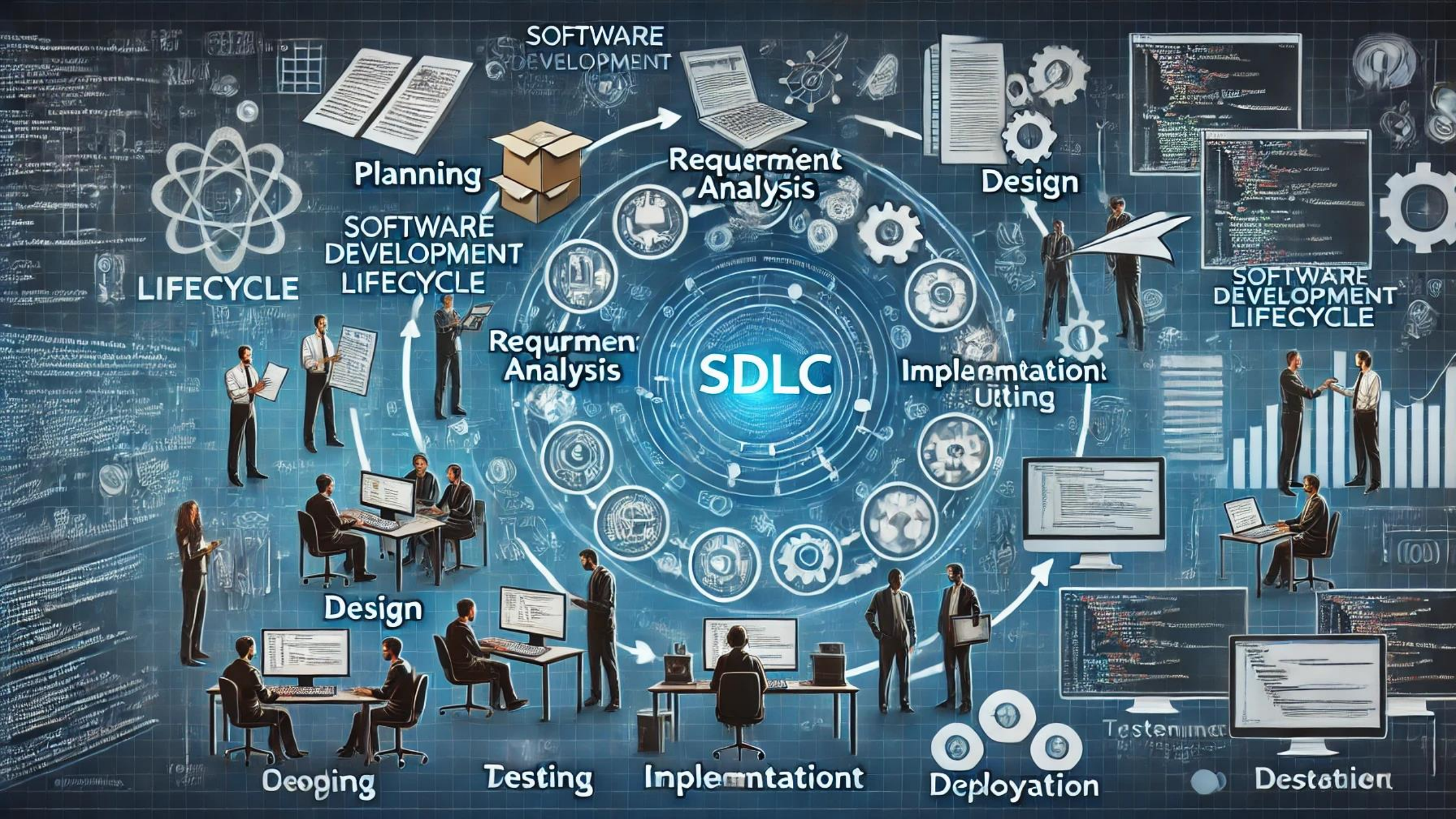
A Comprehensive Guide for Software Engineering

COP 4934
University of Central Florida
Rick Leinecker

What is Requirements Gathering?

- Definition: The process of identifying, documenting, and managing the needs and constraints of stakeholders for a software project.
- Importance: Ensures the software meets user needs, reduces rework, and aligns with business goals.
- Role in the Software Development Lifecycle (SDLC): Bridges the gap between stakeholders and developers.




SOFTWARE DEVELOPMENT
LIFECYCLE
Planning
Requerment Analysis
Design
SOFTWARE DEVELOPMENT LIFECYCLE
SDLC
Requrmen Analysis
Implementation
Design
Deploying
Testing
Implementation
Deployation

- Planning: Define project goals, scope, resources, and timeline.
- Requirements Analysis: Gather and analyze requirements from stakeholders to define detailed functionality.
- Design: Create system and software design documents to specify system architecture and software details.
- Implementation: Develop the software according to design documents using coding standards and practices.
- Testing: Execute code to identify defects and verify functionality meets requirements.
- Deployment: Deploy software to a production environment where it is accessible to users.
- Maintenance: Provide ongoing support, fix bugs, and update software as needed.
- Evaluation: Assess system performance and functionality to ensure it meets business and user needs.
- Documentation: Prepare and maintain documents that describe system features, maintenance procedures, and user guides.
- Iteration: Depending on the project, return to earlier phases for updates or new features based on user feedback and system evaluations.


PROJECT TIMELINE

The Importance of Requirements

- Alignment with Stakeholder Needs: Ensures the software solves the right problems.
- Reduces Risks: Minimizes misunderstandings and costly changes later in the project.
- Provides Clarity: Acts as a roadmap for developers, testers, and project managers.
- Example: A poorly defined requirement can lead to a feature that doesn’t meet user expectations, resulting in wasted time and resources.


SOFTWARE DEVELOPMENT
SOFTWARE DEVELOPMENT
SOFTWARE REQUIREMENT
SOFTWARE DEVELOPMENT
FAILED
FAILED
?
?
?
?
?
?

Components of Software Requirements

- Functional Requirements: What the system should do (e.g., "The system must generate monthly reports").
- Non-Functional Requirements: How the system should perform (e.g., "The system must be available 99.9% of the time").
- Constraints: Limitations on the project (e.g., "The system must use Java for development").
- Assumptions: Beliefs about the system or environment (e.g., "Users will have basic computer skills").
- Dependencies: External factors the project relies on (e.g., "The system depends on a third-party payment gateway").

Identifying Stakeholders

- Who Are Stakeholders? Individuals or groups with an interest in the project.
- ***Examples:***
- **Clients**: Pay for the software.
- **End-Users**: Will use the software.
- **Developers**: Build the software.
- **Managers**: Oversee the project.
- **Importance**: Each stakeholder group may have different needs and priorities.
- ***Tip: Create a stakeholder map to identify and categorize stakeholders*.**

Common Challenges

- **Miscommunication**: Stakeholders and developers may use different terminology.
- **Changing Requirements**: Needs may evolve during the project.
- **Conflicting Needs**: Different stakeholders may have competing priorities.
- **Lack of Clarity**: Vague or incomplete requirements can lead to misunderstandings.
- **Example**: A stakeholder says, "Make it user-friendly," but doesn’t define what that means.

Techniques for Gathering Requirements

- Overview: Different methods to collect requirements based on the project context.
- Techniques:
- Interviews
- Surveys and Questionnaires
- Workshops
- Observation
- Document Analysis
- Prototyping
- Brainstorming
- ***Tip: Use a combination of techniques for the best results.***
- ***Also consider UML Diagrams***


INTERVIEWS
Surveys and Questionnaires
Surveys


INTERVIEWS


REQUREMENTS


PROTOSROMING


DOCUMENT ASSEFIFATIONS

Conducting Interviews

- What Are Interviews? One-on-one or group discussions with stakeholders.
- Types:
- Structured: Predefined questions.
- Unstructured: Open-ended conversations.
- Tips:
- Prepare a list of questions in advance.
- Listen actively and ask follow-up questions.
- Record and document the responses.
- ***Example Question: "What are the most critical features you need in this system?"***

Using Surveys and Questionnaires

- When to Use: When you need input from a large group of stakeholders.
- Design Tips:
- Use clear and concise questions.
- Include multiple-choice, rating scales, and open-ended questions.
- Pilot test the survey before distributing it.
- Pros: Scalable, cost-effective.
- Cons: Limited depth of responses.
- ***Example: A survey to gather feedback on a new feature idea.***

Facilitating Workshops

- What Are Workshops? Collaborative sessions with multiple stakeholders.
- Benefits:
- Encourages teamwork and creativity.
- Provides immediate feedback.
- Tips:
- Set clear objectives for the workshop.
- Use visual aids like whiteboards or sticky notes.
- Assign a facilitator to guide the discussion.
- ***Example: A workshop to define the user interface design.***

Observing Users

- What Is Observation? Watching users interact with a system or perform tasks.
- Benefits:
- Provides real-world insights into user behavior.
- Identifies pain points and inefficiencies.
- Challenges:
- Time-consuming.
- Users may behave differently when observed.
- ***Example: Observing a cashier using a point-of-sale system to identify usability issues.***

Analyzing Existing Documents

- What Is Document Analysis? Reviewing existing materials to extract requirements.
- Examples of Documents:
- Business process manuals.
- Legacy system documentation.
- Regulatory requirements.
- Benefits: Saves time and provides historical context.
- ***Example: Reviewing a company's HR manual to identify requirements for a new employee management system.***

Creating Prototypes

- What Is Prototyping? Building a preliminary version of the software.
- Types:
- Low-Fidelity: Sketches or wireframes.
- High-Fidelity: Interactive mockups.
- Benefits:
- Clarifies requirements through visualization.
- Provides early feedback from stakeholders.
- ***Example: A clickable prototype of a mobile app to demonstrate navigation flow.***

Brainstorming Ideas

- What Is Brainstorming? A group activity to generate creative ideas.
- Rules:
- Encourage free thinking.
- Avoid criticism during the session.
- Build on others' ideas.
- Tips:
- Use techniques like mind mapping.
- Prioritize ideas after the session.
- ***Example: Brainstorming features for a new e-commerce platform.***

Writing Use Cases and User Stories

- Use Cases: Describe how users interact with the system to achieve a goal.
- ***Example: "As a user, I want to reset my password so that I can regain access to my account."***
- User Stories: Short, simple descriptions of a feature from the user’s perspective.
- Format: "As a [role], I want [feature] so that [benefit]."
- Benefits: Easy to understand and prioritize.

Defining Functional Requirements

- What Are Functional Requirements? Descriptions of what the system should do.
- Examples:
- "The system must allow users to create an account."
- "The system must generate a monthly sales report."
- Tips:
- Use clear and specific language.
- Avoid ambiguity.
- ***Example Template: "The system shall [action]."***

Defining Non-Functional Requirements

- Performance: "The system must handle 10,000 concurrent users."
- Security: "The system must encrypt user passwords."
- Usability: "The system must have a response time of less than 2 seconds."
- Tips:
- Use measurable criteria.
- Prioritize based on stakeholder needs.

Identifying Constraints and Assumptions

- Constraints: Limitations on the project (e.g., budget, timeline, technology).
- ***Example: "The system must be compatible with iOS and Android."***
- Assumptions: Beliefs about the system or environment.
- ***Example: "Users will have access to high-speed internet."***
- Importance: Helps set realistic expectations and guides decision-making.

Prioritizing Requirements

- Why Prioritize? Ensures the most critical needs are addressed first.
- Techniques:
- MoSCoW Method: Must-have, Should-have, Could-have, Won’t-have.
- Kano Model: Classifies features based on customer satisfaction.
- ***Example: A "Must-have" requirement for an e-commerce site is a secure payment gateway.***

Documenting Requirements

- Why Document? Provides a clear and shared understanding of the project.
- Tools:
- Requirements Specification Documents.
- Use Case Diagrams.
- User Story Maps.
- Best Practices:
- Use clear and concise language.
- Include examples and diagrams.
- Regularly update the documentation.

Ensuring Traceability

- What Is Traceability? The ability to track requirements throughout the project lifecycle.
- Why Is It Important? Ensures all requirements are implemented and tested.
- Tools: Traceability matrices, requirements management software.
- ***Example: Linking a requirement to its corresponding test case.***

Validating Requirements

- What Is Validation? Ensuring requirements meet stakeholder needs.
- Techniques:
- Reviews: Stakeholders review and approve requirements.
- Prototyping: Demonstrates functionality.
- Testing: Verifies requirements are met.
- ***Example: Conducting a review session with stakeholders to confirm requirements*.**

Common Mistakes in Requirements Gathering

- Incomplete Requirements: Missing key details.
- Vague Language: Using terms like "user-friendly" without definition.
- Ignoring Non-Functional Requirements: Focusing only on features.
- ***Example: A project fails because security requirements were overlooked.***
- Slide 25: Tools for Requirements Gathering
- Title: Tools and Software
- Content:
- Overview of Tools:
- JIRA: For tracking requirements and tasks.
- Confluence: For documentation.
- Trello: For visual task management.
- How to Choose: Consider project size, team preferences, and budget.

Agile Requirements Gathering

- How Agile Differs: Iterative and incremental approach.
- Role of the Product Owner: Manages the product backlog and prioritizes requirements.
- ***Example: Writing user stories for each sprint in Scrum*.**

Case Study: Successful Requirements Gathering

- Example Project: A mobile banking app.
- Key Factors: Clear communication, stakeholder involvement, thorough documentation.
- Outcome: The app met all user needs and was delivered on time.

Case Study: Failed Requirements Gathering

- Example Project: A healthcare management system.
- Key Issues: Incomplete requirements, lack of stakeholder input.
- Outcome: The system failed to meet user needs, leading to costly rework.

Best Practices for Requirements Gathering

- Involve all stakeholders early.
- Use multiple techniques.
- Document and validate requirements thoroughly.
- Prioritize and manage changes effectively.


Done Gathering
Requirements

ANY
QUESTIONS?

Any
questions?